Actividad práctica en parejas

Objetivo: tomar una pelea por turnos en consola que tiene cuatro problemas de comportamiento, decidir qué patrón corresponde a cada uno y justificarlo, y después aplicar State, Strategy y Command. Quien termine antes agrega una táctica nueva y aplica también Observer.

Los archivos

El código de la actividad está en el repositorio de la cátedra. Si ya está clonado, alcanza con actualizarlo desde la carpeta del repositorio:

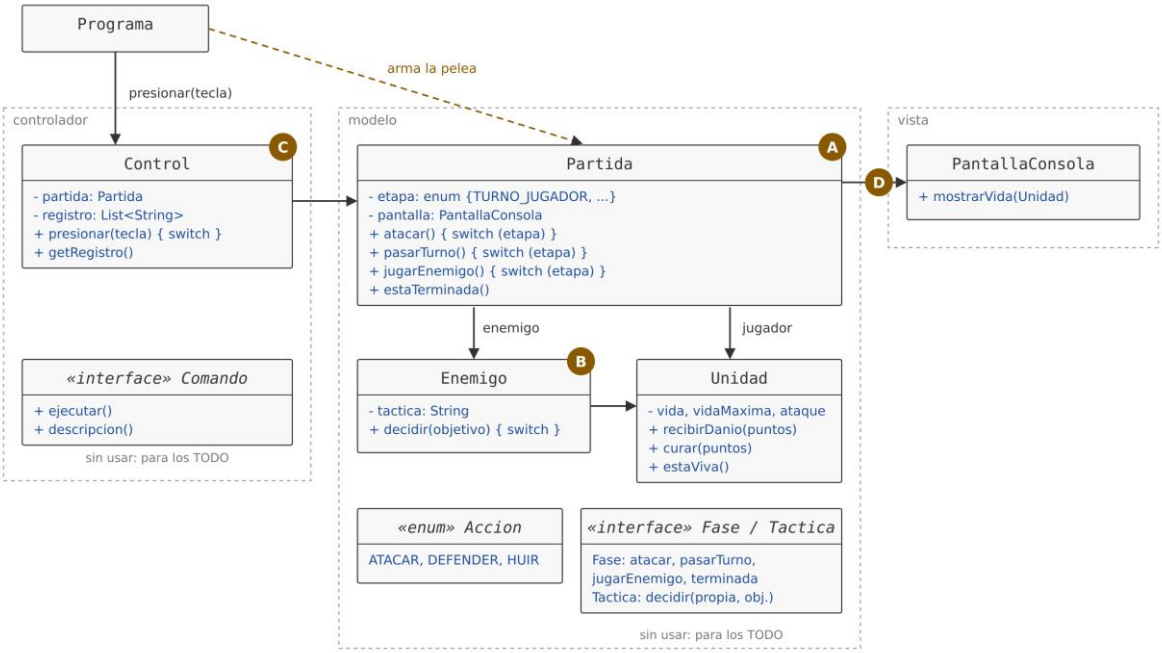
```
git pull
```

Si no, se clona una única vez:

```
git clone https://github.com/POO-CR/poo-2026.git
```

También está en archivo comprimido en el classroom. La carpeta de hoy es *15-comportamiento/actividad*.

Es una pelea entre el jugador y un enemigo. *Unidad* es un personaje con vida y ataque. *Enemigo* es una unidad con una táctica, que hoy es un texto. *Partida* controla los turnos, el del jugador, el del enemigo y el final, con *atacar()*, *pasarTurno()* y *jugarEnemigo()*. *Control*, en el paquete *controlador*, es el panel del jugador: recibe una tecla, hace la acción y la anota en el registro de combate. *PantallaConsola*, en el paquete *vista*, muestra la vida de una unidad. *Fase*, *Tactica* y *Comando* son tres interfaces que todavía no se usan, preparadas para los TODO. *Programa* arma una pelea entre Aragorn y un troll con táctica defensiva, la juega con el control hasta el final e imprime el registro.



El código de partida. Las letras marcan dónde está cada problema, y las tres interfaces todavía no se usan.

Parte 1: decidir

Ejecutar *Programa* y leer la salida ronda por ronda. Después buscar en el código los comentarios “Problema A”, “Problema B”, “Problema C” y “Problema D”.

- Problema A, en *Partida*. Las tres operaciones tienen un *switch* sobre la etapa de la partida, y cada rama decide si la operación se permite y a qué etapa se pasa.
- Problema B, en *Enemigo*. *decidir()* tiene un *switch* sobre el nombre de la táctica, que eligió quien armó el nivel.
- Problema C, en *Control*. *presionar()* tiene un *switch* sobre la tecla, y cada rama llama a la partida y escribe su propia entrada del registro. Cambiar una tecla o agregar una acción obliga a abrir *Control*.
- Problema D, en *Partida*. Cada vez que la vida de una unidad cambia, *Partida* llama a la pantalla. El paquete *modelo* importa una clase de *vista*.

Para cada problema, anotar en la pareja qué patrón de los cuatro vistos corresponde, Observer, State, Strategy o Command, y justificarlo en una o dos oraciones respondiendo qué se convierte en objeto y quién lo elige o lo cambia. Con la tabla completa, seguir.

Parte 2: State, Strategy y Command

Un TODO por vez, en orden. Cada TODO indica en qué archivos se trabaja. Entre el TODO 1 y el TODO 2 el proyecto no compila: es esperable.

- TODO 1 (tres clases nuevas en *modelo*): *TurnoJugador*, *TurnoEnemigo* y *Terminada*, que implementan *Fase*. Cada una decide qué operaciones permite, devolviendo *false* en

las que no, y hace la transición llamando a *partida.cambiarFase()*. Para hacer la acción en sí usan los métodos que ya tiene *Partida*, *ejecutarAtaqueJugador()* y *ejecutarAccionEnemigo()*. Las reglas son las mismas que hoy tienen los *switch*, así que alcanza con leerlas rama por rama y pasar cada una a su clase. Los métodos que las fases usan y todavía no existen se agregan a *Partida* en el TODO 2: *cambiarFase(Fase nueva)*, *jugadorVivo()* y *enemigoVivo()*.

- TODO 2 (*Partida.java*): el *enum* y los tres *switch* desaparecen. *Partida* guarda una *Fase*, que empieza en *new TurnoJugador()*, y cada operación pública delega en ella. *estaTerminada()* también. Se agregan *cambiarFase(Fase nueva)*, *jugadorVivo()* y *enemigoVivo()*, y junto con los dos métodos de las acciones quedan sin modificador de acceso, así los ven las fases pero nadie de afuera del paquete.
- TODO 3 (clases nuevas, *Enemigo.java*, *Programa.java*): *TacticaAgresiva* y *TacticaDefensiva* implementan *Tactica* con las reglas del *switch* de *Enemigo*. *Enemigo* recibe una *Tactica* en vez de un texto, también en *cambiarTactica()*, y *decidir()* le delega. En *Programa*, el troll se crea con *new TacticaDefensiva()*.
- TODO 4 (clases nuevas en *controlador*, *Control.java*, *Programa.java*): *ComandoAtacar* y *ComandoPasar* implementan *Comando*. Reciben la *Partida* por constructor, *ejecutar()* le pide la acción y devuelve si se pudo, y *descripcion()* devuelve el texto que hoy escribe cada rama del registro. *Control* deja de recibir la partida: guarda un mapa de tecla a *Comando*, con un método *boolean asignar(char tecla, Comando comando)*, y un historial con los comandos que se pudieron ejecutar. *presionar()* busca el comando de la tecla y lo ejecuta, devuelve *false* si la tecla no tiene comando o si el comando no se pudo ejecutar, y *getRegistro()* arma la lista con las descripciones del historial. En *Programa*, después de crear el control, se le asignan la *a* y la *p*.

Con los cuatro TODO hechos, la salida es idéntica a la del principio, registro incluido. Antes de seguir, comprobar que la clase *Control* ya no importa nada de *modelo*. Los comandos sí: son los que conocen a la partida.

Para quien termina antes

- TODO 5, un orco cobarde: agregar *TacticaCobarde*, que ataca mientras tiene al menos el 30% de su vida y huye cuando baja de eso. En *Programa*, después de la primera pelea, armar una segunda entre Boromir, con 50 de vida y 10 de ataque, y un orco con 40 de vida, 6 de ataque y táctica cobarde, con su propia *Partida* y la misma pantalla. Jugarla sin control, llamando a *atacar()* y *jugarEnemigo()* hasta que termine, e imprimir si el orco sigue vivo: tiene que estarlo. Ojo con el 30%: con enteros, *vidaMaxima / 100* 30* da cero. Comprobar que ni *Enemigo* ni *Partida* cambiaron para que entrara la táctica nueva.
- TODO 6, Observer: *Unidad* avisa cuando su vida cambia. En *modelo*, una interfaz *ObservadorUnidad* con *vidaCambio(Unidad unidad)*. *Unidad* guarda una lista de observadores, acepta nuevos con *agregarObservador()* y avisa al final de *recibirDanio()* y de *curar()*. *PantallaConsola* implementa *ObservadorUnidad*. Se avisa solo cuando la vida cambió, es decir, cuando el método va a devolver *true*. *Partida* deja de recibir la pantalla y de llamarla, y *Programa* registra la pantalla en cada unidad, para lo que

conviene sacar la unidad del troll a una variable. El paquete *modelo* ya no importa nada de *vista*.